

Introduction – Sémantique concrète

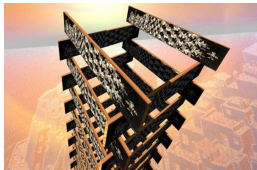
TAS : Typage et analyse statique
M2, Master STL, Sorbonne Université

Antoine Miné

Année 2024–2025

Cours 8

22 novembre 2024



Objectifs du cours : 2ème partie

Vérification de programmes par **analyse statique sûre**
basée sur la théorie de l'**interprétation abstraite**.

- **Maîtriser** les **principes** et l'état de l'art.

(que fait-on ? à quoi cela peut-il servir ?)

- **Comprendre** les **justifications théoriques**.

(pourquoi est-ce que cela marche ? quelles règles faut-il respecter ?)

Quelques théorèmes et quelques preuves ^{*}_{★ ★}

mais sans entrer dans les aspects théoriques les plus techniques.

- Savoir **programmer** une analyse statique sûre simple, sur un langage jouet.
(comment le faire fonctionner en pratique ?)

Implantation d'un **projet** en **TME**, complété à la maison.

Introduction

Analyse statique

source

```
int search(int* t, int n) {
    int i;
    for (i=0; i<n; i++) {
        if (t[i]) break;
    }
    return t[i];
}
```



résultat de l'analyse

```
int search(int* t, int n) {
    int i;
    for (i=0; i<n; i++) {
        //  $0 \leq i < n$ 
        if (t[i]) break;
    }
    //  $0 \leq i \leq n \vee n < 0$ 
    return t[i];
}
```



- analyse le code **source** original (pas de modèle, pas d'instrumentation)
- infère des **propriétés** pour déduire la **correction** des programmes
 au delà du **typage** : propriétés sensibles au flot de contrôle, sur les valeurs ;
 application à la preuve d'absence de *run-time errors* (débordements de tableaux, pointeurs, etc.)
- **statiquement** (à la compilation, sans exécution)
- **automatiquement** (sans interaction avec l'utilisateur)
- de manière **efficace**

⇒ particulièrement adapté à une utilisation industrielle

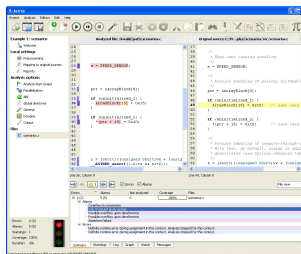
L'inférence des propriétés les plus précises est **un problème indécidable !**

⇒ nous utilisons des **approximations sûres**.

Quelques outils utilisés dans l'industrie

■ Astrée

- analyse de code C embarqué critique
- vérifie l'absence d'erreurs à l'exécution
- spécialisé pour le code avionique
- outil académique, puis industrialisé par AbsInt (2009–)



■ Infer.AI par Facebook

- logiciel libre <http://fbinfer.com/> (2015–)
- plateforme générale d'analyse modulaire pour Java, C, C++, etc.
- cible l'analyse des *merge requests*

Sémantique

Principe : définir un sens mathématique à un programme.

Applications :

- comprendre ce qu'est, fondamentalement, un programme, ce qu'est un calcul ;
- raisonner sur les programmes ;
(en raisonnant sur leur interprétation mathématique)
- justifier la validité des méthodes automatiques de vérification.

Nombreuses variantes : sémantiques opérationnelles, axiomatiques, dénotationnelles, etc.

Validité d'une analyse statique :

- en partant d'une sémantique concrète du langage ;
(base de confiance décrivant minutieusement le comportement des programmes)
- par abstraction successive des opérations du langage
nous oublions les informations *a priori* inutiles à la preuve jusqu'à obtenir une sémantique calculable.

Interprétation abstraite



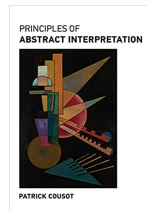
Patrick Cousot



Radhia Cousot



— 1



2

Théorie **générale** de l'**approximation**
et de la **comparaison** des sémantiques :

- permet d'**unifier** des sémantiques disparates ;
 - guide la **conception** de nouvelles analyses statiques
qui sont **sûres par construction** ;
- ⇒ nous l'utiliserons pour concevoir nos analyses.

-
1. P. Cousot. Méthodes itératives de construction et d'approximation de points fixes d'opérateurs monotones sur un treillis, analyse sémantique des programmes. Thèse És Sciences Mathématiques, 1978.
 2. P. Cousot. Principles of Abstract Interpretation. MIT Press, 2021.

À voir également : Colloquium sur la compilation certifiée

Colloquium d'Informatique de Sorbonne Université

**Sandrine
Blazy**

Université de Rennes



**26
nov
2024**

**Compilation vérifiée :
Vers du logiciel zéro défaut**

<https://www.lip6.fr/colloquium>

mardi prochain, 18h, Amphi 25 sur le campus de Jussieu

Organisation du cours

Organisation générale de la 2ème partie

Équipe pédagogique :

- Cours : Antoine Miné
- TD/TME : Milla Valnet

Plan : 2h de cours + 2h de TD/TME à chaque séance

- Séance 8 : Introduction, sémantique concrète
- Séance 9 : Abstraction de sémantiques
- Séance 10 : Analyse de boucles
- Séance 11 : Produits réduits, domaines disjonctifs
- Séance 12 : Domaines relationnels
- Séance 13 : Analyse de pointeurs, analyse de tableaux
- Séance 14 : Application : vérification de logiciels embarqués avioniques

Évaluation

2ème partie du cours TAS = 50% de la note de l'UE, dont :

- **Projet d'analyseur : 25%**
 - projet individuel
 - avancé en TME et à la maison
 - à rendre le jour de l'examen
- **Examen final : 25%**
 - examen écrit de 2h
 - similaire aux TD (en plus long)
 - exemple d'annale corrigée sur Moodle

Ressources :

les transparents sont mis au fur et à mesure sur le **site Moodle** du cours :
<https://moodle-sciences-24.sorbonne-universite.fr>

Projet d'analyseur

But : écrire un petit analyseur statique

- pour un langage jouet (variables entières, affectations, tests if-then-else, boucles)
- en OCaml
- analyse des valeurs numériques des variables
(analyse de constantes, d'intervalles, produit réduit entre intervalles et parités)
- extensions au choix dans la liste proposée

Fourni : dans un GitLab STL privé : <https://stl.algo-prog.info/>

- sources du *front-end* (analyse syntaxique, AST)
- *squelette* d'analyseur (interfaces, itérateur, domaine concret et des constantes)
- banque de tests et scripts d'intégration continue

Travail et rendu :

- création d'un *fork privé* du projet sous GitLab STL
- consignes à suivre dans [README.md](#) et [TRAVAIL.md](#)
- rapport à fournir dans [RAPPORT.md](#) (texte simple ou Markdown)
- le projet GitLab est ramassé sur le serveur le jour de l'examen (faire des pushes !)

Organisation des TME

Séances de TME : développement du projet d'analyseur

- sujet de projet donné dès le 1er TME
- séance de TME = l'occasion de mettre en pratique le cours pour avancer sur le projet
- suggestion de programme :
 - Séance 8 : prise en main, sémantique concrète
 - Séance 9 : domaine des constantes
 - Séance 10 : domaine des intervalles
 - Séance 11 : analyse des boucles
 - Séance 12 : produit réduit
 - Séance 13 : début de développement d'une extension avancée
 - Séance 14 : finalisation du projet
- conseils d'implantation régulièrement donnés à la fin des transparents

Venir en TME permet d'avoir une bonne note au projet !

Langage numérique

Objectifs

Proposer un langage simple :

- dont la sémantique est **facile à écrire** ;
- qui va nous servir à **illustrer l'analyse de propriétés numériques** ;
- proche du langage cible du **projet** d'analyste.

Nous présentons dans ce cours :

- la **syntaxe** du langage ;
- une **sémantique concrète** d'états, **non calculable** ;
- quelques **outils mathématiques** et théorèmes utiles pour justifier que la sémantique est bien fondée ;
- en TME, le **code** calculant la sémantique concrète qui sert de base au projet d'analyste.

Prochains cours : dérivation de sémantiques approchées calculables.

Syntaxe des instructions

Instructions

<i>stat</i>	$::=$	$X \leftarrow expr$	(affectation, $X \in \mathbb{V}$)
		$stat; stat$	(séquence)
		if <i>cond</i> then <i>stat</i> else <i>stat</i>	(test)
		while <i>cond</i> do <i>stat</i>	(boucle)
		assert <i>cond</i>	(vérification de condition)
		halt	(arrêt du programme)
		skip	(ne rien faire)

- présentation sous forme de **grammaire BNF**
- l'ensemble des **variables** $\mathbb{V}$ est fini et fixé (pas d'allocation dynamique)
(dans le projet, les blocs déclareront des variables locales ; ici, elles sont globales)
- les **expressions** *expr* des affectations sont **arithmétiques** ;
les **conditions** *cond* des tests, boucles, assertions sont **booléennes**
(voir transparents suivants)
- **skip** est utile pour programmer le **if – then** sans **else**
- **assert** permet de spécifier les propriétés à prouver

Syntaxe des expressions booléennes

expressions booléennes (conditions)

<i>cond</i>	::=	<i>expr</i> $\bowtie$ <i>expr</i>	(comparaison)
		<i>c</i>	(constantes logiques, $c \in \mathbb{B}$)
		$\neg$ <i>cond</i>	(négation logique)
		<i>cond</i> $\wedge$ <i>cond</i>	(et logique)
		<i>cond</i> $\vee$ <i>cond</i>	(ou logique)

- **comparaisons** d'expressions entières : $\bowtie \in \{\leq, \geq, =, \neq, <, >\}$
- **constantes logiques** : $c \in \mathbb{B}$ où $\mathbb{B} \stackrel{\text{def}}{=} \{\text{true}, \text{false}\}$
- **opérations logiques** classiques : $\vee, \wedge, \neg$
- pour simplifier, les variables ne peuvent pas contenir de booléens
(et n'apparaissent donc pas directement dans les conditions)

Syntaxe des expressions arithmétiques

expressions arithmétiques

$expr$	$::=$	X	(variable, $X \in \mathbb{V}$)
		c	(constante, $c \in \mathbb{Z}$)
		$\diamond expr$	(opération unaire)
		$expr \diamond expr$	(opération binaire)
		$\mathbf{rand}(c_1, c_2)$	(valeur aléatoire entre deux constantes)

- les **constantes** et **variables** sont à valeur dans $\mathbb{Z}$
(utilisation des entiers mathématiques illimités, pas les entiers machine, pour simplifier)
- $\diamond$ dénote les **opérations arithmétiques** classiques :
 - opération unaire : $-$
 - opérations binaires : $+$, $-$, $\times$, $/$
- **rand**(c_1, c_2) modélise un **choix non-déterministe**
entre deux valeurs constantes : $c_1 \in \mathbb{Z} \cup \{-\infty\}$, $c_2 \in \mathbb{Z} \cup \{+\infty\}$
(voir transparent suivant)

Justification du non-déterminisme

Le **non-déterminisme** dans un programme permet de :

- modéliser un environnement partiellement **inconnu**
(entrées de l'utilisateur, fichiers, communications réseau, ...)
- abstraire des **parties inconnues** du programme (bibliothèques)
- abstraire des **parties trop complexes** du programme (arrondi flottant)
- analyser en une fois une **famille de programmes** (programme paramétré)

Syntaxe :

Non-déterminisme sur les données : $V \leftarrow \mathbf{rand}(0, 255)$.

Le non-déterminisme sur le contrôle peut s'écrire : « **if** $\mathbf{rand}(0, 1) = 0$ **then** s_1 **else** s_2 »

Impact sur la sémantique et la vérification

il nous faut vérifier **toutes** les exécutions possibles

(quelles que soient les entrées, l'implantation des bibliothèques, les paramètres du programme, ...)

⇒ la sémantique doit donc énumérer **tous** les cas (il y en a un nombre infini...).

Exemple de programme non-déterministe

Exemple

```
Y ← rand(0, 100);  
X ← 0;  
while X ≤ Y do  
    X ← X + rand(0, 10)  
done;  
assert X ≤ 100
```

- l'**assertion** est-elle vérifiée ?
- quelle est la valeur de **X** à la fin du programme ?
(après l'assertion)

Quizz : $\text{rand}(0, 1) = \text{rand}(0, 1)$ est-il vrai ou faux ?

Choix d'une sémantique concrète collectrice

Sémantique concrète collectrice :

- définition mathématique **précise** du comportement du programme, servant de **base** à la construction de l'analyse ;
la sûreté de l'analyse abstraite est prouvée vis à vis de cette sémantique
- choisie pour exprimer les propriétés (**non décidables**) d'intérêt de l'analyse, mais pas plus !

A priori, plusieurs choix sont possibles :

- sémantique des **traces d'exécution** :

$$\begin{aligned} &\{(\textcircled{1}, X = 0, I = 0) \rightarrow (\textcircled{2}, X = 100, I = 0) \\ &\quad \rightarrow (\textcircled{3}, X = 100, I = 1) \rightarrow (\textcircled{3}, X = 100, I = 4) \\ &\quad \rightarrow (\textcircled{3}, X = 100, I = 7) \rightarrow (\textcircled{3}, X = 100, I = 10) \rightarrow \dots ; \\ &(\textcircled{1}, X = 0, I = 0) \rightarrow (\textcircled{2}, X = 101, I = 0) \rightarrow \dots ; \\ &\dots\} \end{aligned}$$

- sémantique des **états accessibles** :

$$\begin{aligned} \textcircled{1} : & \{(X = 0, I = 0)\} \\ \textcircled{2} : & \{(X = 100, I = 0), (X = 101, I = 0), \dots\} \\ \textcircled{3} : & \{(X = 100, I = 1), (X = 100, I = 4), \dots \\ & \quad (X = 101, I = 1), (X = 101, I = 4), \dots\} \end{aligned}$$

abstraction de la sémantique de traces !

Pour prouver la sûreté de fonctionnement, les états accessibles suffisent.
(les traces seraient nécessaires pour prouver la terminaison)

Exemple

```

① X ← rand(100,200);
② I ← 1;
while I ≤ X do
    ③ I ← I + 3
done
  
```

Sémantique « à grands pas »

Notre choix : **sémantique « à grands pas »**

- **fonction** (relation) qui aux **états mémoires en entrée** du programme
- associe les **états mémoires possibles en sortie** du programme
- sans garder les états intermédiaires.

C'est donc une **abstraction** de la sémantique d'états accessibles.

Avantages :

- plus efficace en mémoire ;
- programmable facilement **par induction sur la syntaxe**
 $\implies$ l'analyseur ressemble à un interprète classique ;
- sémantique des **triplets de Hoare**.

Exemple

```
X ← 100;
I ← 1;
while I ≤ X do
    I ← I + 3
done
```

- **sémantique fonctionnelle :**
 $(X = x, I = i) \mapsto (X = 100, I = 103)$

Quelles que soient les valeurs x de X et i de I en entrée, nous avons $X = 100$ et $I = 103$ en sortie.

Les états intermédiaires sont oubliés.

En pratique nous gardons quand même une trace des erreurs possibles à l'exécution, par exemple en les affichant au fur et à mesure de l'analyse.

Sémantique

Environnements, sémantique des expressions

$$\underline{E[\textit{expr}]} : \mathcal{E} \rightarrow \mathcal{P}(\mathbb{Z})$$

- les environnements $\mathcal{E} \stackrel{\text{def}}{=} \mathbb{V} \rightarrow \mathbb{Z}$
donnent une valeur dans $\mathbb{Z}$ à chaque variable
(c'est l'état mémoire du programme)
- $E[\textit{expr}] \rho$ est l'évaluation de l'expression $\textit{expr}$
dans l'environnement $\rho \in \mathcal{E}$.
Elle retourne un **ensemble de valeurs** dans $\mathbb{Z}$
à cause du **non-déterminisme** de **rand**.

$$\begin{array}{ll} E[V] \rho & \stackrel{\text{def}}{=} \{\rho(V)\} \\ E[c] \rho & \stackrel{\text{def}}{=} \{c\} \\ E[\text{rand}(a, b)] \rho & \stackrel{\text{def}}{=} \{x \in \mathbb{Z} \mid a \leq x \leq b\} \end{array}$$

Notation : les notations de la forme $X[y]$ seront souvent utilisées ;
 y dénote un objet syntaxique ;
 X distingue des fonctions sémantiques avec des signatures différentes (expressions, instructions, etc.) ;
 $X[y] z$ dénote l'application de la fonction $X[y]$ à la valeur z .

Sémantique des expressions (suite)

$E\llbracket \text{expr} \rrbracket : \mathcal{E} \rightarrow \mathcal{P}(\mathbb{Z})$

- définie par **induction structurelle** sur la syntaxe de l'expression
- nous pouvons avoir $E\llbracket e \rrbracket \rho = \emptyset$ à cause d'une division par zéro
- la valeur $\emptyset$ se propage ;
ainsi $E\llbracket 1 + (2/0) \rrbracket \rho = \emptyset$ car $E\llbracket 2/0 \rrbracket \rho = \emptyset$.

$$\begin{aligned}
 E\llbracket -e \rrbracket \rho &\stackrel{\text{def}}{=} \{ -v \mid v \in E\llbracket e \rrbracket \rho \} \\
 E\llbracket e_1 + e_2 \rrbracket \rho &\stackrel{\text{def}}{=} \{ v_1 + v_2 \mid v_1 \in E\llbracket e_1 \rrbracket \rho, v_2 \in E\llbracket e_2 \rrbracket \rho \} \\
 E\llbracket e_1 - e_2 \rrbracket \rho &\stackrel{\text{def}}{=} \{ v_1 - v_2 \mid v_1 \in E\llbracket e_1 \rrbracket \rho, v_2 \in E\llbracket e_2 \rrbracket \rho \} \\
 E\llbracket e_1 \times e_2 \rrbracket \rho &\stackrel{\text{def}}{=} \{ v_1 \times v_2 \mid v_1 \in E\llbracket e_1 \rrbracket \rho, v_2 \in E\llbracket e_2 \rrbracket \rho \} \\
 E\llbracket e_1 / e_2 \rrbracket \rho &\stackrel{\text{def}}{=} \{ v_1 / v_2 \mid v_1 \in E\llbracket e_1 \rrbracket \rho, v_2 \in E\llbracket e_2 \rrbracket \rho \setminus \{0\} \}
 \end{aligned}$$

Quizz : que valent $E\llbracket \text{rand}(-1, 1)/0 \rrbracket \rho$ et $E\llbracket 1/\text{rand}(-1, 1) \rrbracket \rho$?

Sémantique des conditions booléennes

$$\underline{C[\textit{cond}] : \mathcal{P}(\mathcal{E}) \rightarrow \mathcal{P}(\mathcal{E})}$$

- notre sémantique manipule des **ensembles d'environnements** ;
- les conditions agissent comme des filtres ;
elles **sélectionnent** les états mémoires qui satisfont la condition et peuvent continuer l'exécution du programme.

$$C[\textit{true}] R \stackrel{\text{def}}{=} R$$

$$C[\textit{false}] R \stackrel{\text{def}}{=} \emptyset$$

$$C[c_1 \wedge c_2] R \stackrel{\text{def}}{=} C[c_1] R \cap C[c_2] R$$

$$C[c_1 \vee c_2] R \stackrel{\text{def}}{=} C[c_1] R \cup C[c_2] R$$

$$C[e_1 = e_2] R \stackrel{\text{def}}{=} \{ \rho \in R \mid \exists v_1 \in E[e_1] \rho, v_2 \in E[e_2] \rho : v_1 = v_2 \}$$

$$C[e_1 < e_2] R \stackrel{\text{def}}{=} \{ \rho \in R \mid \exists v_1 \in E[e_1] \rho, v_2 \in E[e_2] \rho : v_1 < v_2 \}$$

...

Sémantique des conditions booléennes (suite)

- La négation $\neg$ peut être gérée par **transformation d'expression** :

$$\begin{array}{ll} \neg(e_1 = e_2) \rightarrow e_1 \neq e_2 & \neg(c_1 \vee c_2) \rightarrow (\neg c_1) \wedge (\neg c_2) \\ \neg(e_1 < e_2) \rightarrow e_1 \geq e_2 & \neg(c_1 \wedge c_2) \rightarrow (\neg c_1) \vee (\neg c_2) \end{array}$$

...

$\neg$ est éliminé par lois de De Morgan

- On peut vérifier que :

- $C\llbracket cond \rrbracket \{\rho\}$ est soit $\{\rho\}$, soit $\emptyset$;
- $C\llbracket cond \rrbracket R = \bigcup_{\rho \in R} C\llbracket cond \rrbracket \{\rho\}$ (morphisme complet pour l'union)
 $\implies$ chaque environnement est filtré indépendamment des autres.

Quizz : que se passe-t-il en cas d'erreur dans l'expression ?

- $C\llbracket 1 = (1/0) \rrbracket R$
- $C\llbracket true \vee (1 = 1/0) \rrbracket R$

Domaine sémantique des instructions

$$\underline{S[\![stat]\!] : \mathcal{P}(\mathcal{E}) \rightarrow \mathcal{P}(\mathcal{E})}$$

- Notre sémantique manipule des **ensembles d'environnements**.
- $Q = S[\![stat]\!] P$ signifie :
 - si l'état mémoire est dans P **avant** d'exécuter $stat$
 - alors l'état mémoire est dans Q **après** avoir exécuté $stat$
- $S[\![stat]\!] R = \bigcup_{\rho \in R} S[\![stat]\!] \{\rho\}$
 la signature $\mathcal{P}(\mathcal{E}) \rightarrow \mathcal{P}(\mathcal{E})$ permet de composer facilement les $S[\![stat]\!]$
 $\implies$ simplifie l'écriture des fonctions
- $S[\![stat]\!] \{\rho\} = \emptyset$ si l'exécution de $stat$ sur ρ
 - rencontre un erreur
 - ou boucle indéfiniment.

Quizz : quelle est la sémantique de $X \leftarrow 0; \text{while rand}(0, 1) = 0 \text{ do skip done}$?

Sémantique des instructions

$$\underline{S \llbracket \text{stat} \rrbracket : \mathcal{P}(\mathcal{E}) \rightarrow \mathcal{P}(\mathcal{E})}$$

- **skip** : ne fait rien

$$S \llbracket \text{skip} \rrbracket R \stackrel{\text{def}}{=} R$$

- **halt** : arrête le programme

$$S \llbracket \text{halt} \rrbracket R \stackrel{\text{def}}{=} \emptyset$$

- **affectation** : évalue l'expression et met à jour l'environnement

$$S \llbracket X \leftarrow e \rrbracket R \stackrel{\text{def}}{=} \{ \rho[X \mapsto v] \mid \rho \in R, v \in E \llbracket e \rrbracket \rho \}$$

- pour chaque environnement ρ
- pour chaque valeur v de l'expression, dans $E \llbracket e \rrbracket \rho$
(non-déterminisme)
- génère un nouvel environnement $\rho[X \mapsto v]$ où X est mis à jour

$f[x \mapsto y]$ dénote la fonction qui associe y à x , et $f(z)$ à tout $z \neq x$

Sémantique des instructions (suite)

$$\underline{S[\textit{stat}] : \mathcal{P}(\mathcal{E}) \rightarrow \mathcal{P}(\mathcal{E})}$$

- **séquence** : composition de fonctions

$$S[s_1; s_2] R \stackrel{\text{def}}{=} S[s_2] (S[s_1] R)$$

- **test** : filtrage par la condition et exécution des branches

$$S[\textit{if } c \textit{ then } s_1 \textit{ else } s_2] R \stackrel{\text{def}}{=} S[s_1] (C[c] R) \cup S[s_2] (C[\neg c] R)$$

- filtre les environnements par c
- exécute **les deux** branches s_1 et s_2 **indépendamment**
- **union** des comportements des deux branches avec $\cup$

- **assert** : filtrage par une condition

$$S[\textit{assert } c] R \stackrel{\text{def}}{=} C[c] R$$

et affiche une erreur à l'écran si la condition n'est pas toujours vraie ;
i.e., si $C[\neg c] R \neq \emptyset$.

Sémantique des boucles

Calcul de : $S[\text{while } c \text{ do } s] R$

1 calculer un **invariant de boucle** I

i.e, les environnements atteignables après zéro, un ou plusieurs tours de boucle, au point : **while** • c **do** s

2 nous avons ensuite : $S[\text{while } c \text{ do } s] R = C[\neg c] I$

Le calcul de I peut se faire par un **principe d'induction**

si $\rho \in I$, alors :

- soit il s'agit du premier tour de boucle : $\rho \in R$;
- sinon, ρ est dans l'image de I par un tour de boucle :
 $\rho \in S[s](C[c] I)$

Soit donc à résoudre $I = F(I)$ où $F(X) = R \cup S[s](C[c] X)$.

La **théorie de l'ordre** justifie que :

- cette équation a au moins une solution ;
- elle a exactement une plus petite (donc meilleur) solution ;

$\text{lfp } F$: le **plus petit point fixe** de F

$\implies$ c'est l'invariant **optimal** recherché.

Éléments de théorie de l'ordre

Ordres partiels

Ordres partiels

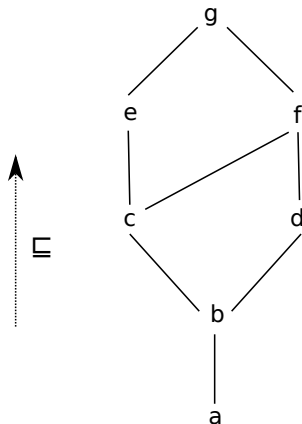
Étant donné un ensemble X ,
la **relation** $\sqsubseteq \in X \times X$ est un **ordre partiel** si elle est :

- 1 réflexive : $\forall x \in X, x \sqsubseteq x$
- 2 transitive : $\forall x, y, z \in X, (x \sqsubseteq y) \wedge (y \sqsubseteq z) \implies (x \sqsubseteq z)$
- 3 anti-symétrique : $\forall x, y \in X, (x \sqsubseteq y) \wedge (y \sqsubseteq x) \implies (x = y)$

$(X, \sqsubseteq)$ est un **poset** (*partially ordered set*).

Exemples d'ordres partiels

- Donné graphiquement par un **diagramme de Hasse**, e.g. :



$g \sqsubseteq g$
 $f \sqsubseteq f, g$
 $e \sqsubseteq e, g$
 $d \sqsubseteq d, f, g$
 $c \sqsubseteq c, e, f, g$
 $b \sqsubseteq b, c, d, e, f, g$
 $a \sqsubseteq a, b, c, d, e, f, g$

Exemples d'ordres partiels

Ordres partiels :

■ $(\mathbb{Z}, \leq)$

(ordre total)

■ $(\mathcal{P}(X), \subseteq)$

(ordre non total : $\{1\} \not\subseteq \{2\}$, $\{2\} \not\subseteq \{1\}$)

■ $(\mathbb{Z}^2, \sqsubseteq)$, où $(a, b) \sqsubseteq (a', b') \iff (a \geq a') \wedge (b \leq b')$

(ordre des bornes d'intervalles pour avoir l'inclusion)

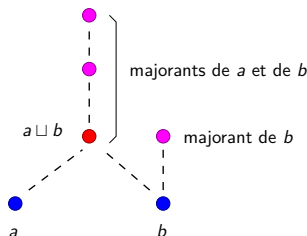
Utilisation des ordres partiels (informellement)

Les **ordres partiels** permettent de modéliser :

- la **logique** : l'implication $\implies$ est un ordre partiel
- les **approximations** : $\sqsubseteq$ donne le degré d'information
(« $a \sqsubseteq b$ » signifie : « a a plus d'information que b »)
- la **vérification de programmes** :
sémantique du programme $\sqsubseteq$ spécification
(e.g. : comportements du programme $\sqsubseteq$ comportements acceptés)
- les **itérations** de point fixe
(le calcul est dirigé et a une limite : $X_1 \sqsubseteq X_2 \sqsubseteq \dots \sqsubseteq X_n$)

(Plus petit) majorant

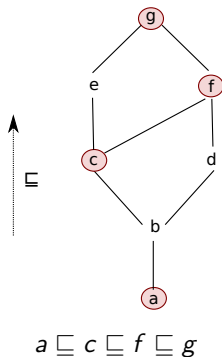
- c est un **majorant** (*upper bound*) de a et de b si : $a \sqsubseteq c$ et $b \sqsubseteq c$
- c est un **plus petit majorant** (*least upper bound*, *lub* ou *join*) de a et b si
 - c est un majorant de a et de b
 - pour tout majorant d de a et b , $c \sqsubseteq d$



- si elle existe, la lub est unique, et notée $a \sqcup b$
- généralisable aux lubs d'ensembles arbitraires $\sqcup Y$
- notion **duale** de (**plus grand**) **minorant** $\sqcap$ (*glb*, *meet*)

Chaînes

$C \subseteq X$ est une **chaîne** de $(X, \sqsubseteq)$ si elle est **totale**ment ordonnée par $\sqsubseteq$:
 $\forall x, y \in C, (x \sqsubseteq y) \vee (y \sqsubseteq x)$.



Ordre partiel complet (CPO)

Un ordre partiel $(X, \sqsubseteq)$ est **complet** (**CPO**) si toute chaîne C a un plus petit majorant $\sqcup C$.

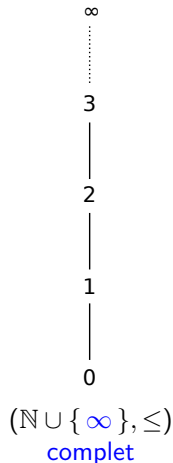
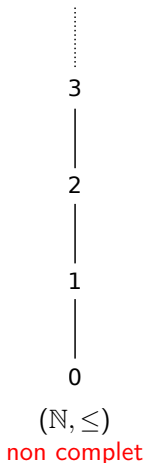
Exemples :

- $(X, \sqsubseteq)$ est complet si X est fini.
- $(\mathcal{P}(Y), \subseteq)$ est complet pour tout Y (même infini).

Ce plus petit majorant correspond à la notion de **limite** : dans un CPO, toute séquence croissante a une limite dans le CPO.

Un CPO a un **plus petit élément** $\sqcup \emptyset$, noté $\perp$.

Ordres complets et incomplets



Par exemple, $\sqcup \{2x \mid x \in \mathbb{N}\}$ existe dans $\mathbb{N} \cup \{\infty\}$, mais pas dans $\mathbb{N}$.

Treillis

Treillis

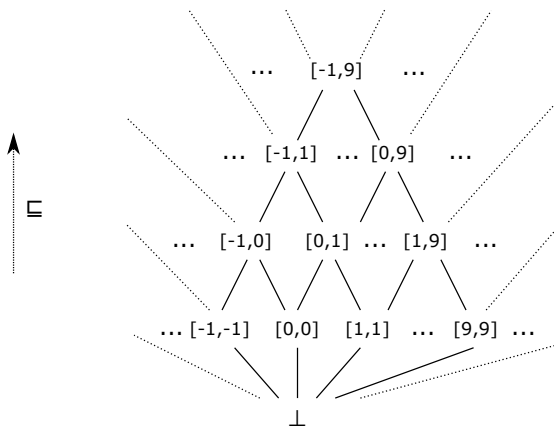
Un **treillis** $(X, \sqsubseteq, \sqcup, \sqcap)$ est un ordre partiel avec

- 1 une lub $a \sqcup b$ pour toute paire d'éléments a, b ;
- 2 une glb $a \sqcap b$ pour toute paire d'éléments a, b .

Exemples :

- les entiers $(\mathbb{Z}, \leq, \max, \min)$
- les intervalles d'entiers (transparent suivant)

Exemple : les intervalles d'entiers



Intervalles d'entiers : $(\{ [a, b] \mid a, b \in \mathbb{Z}, a \leq b \} \cup \{ \perp \}, \subseteq, \sqcup, \cap)$

où $[a, b] \subseteq [a', b'] \stackrel{\text{def}}{\iff} (a \geq a') \wedge (b \leq b')$, $\forall X: \perp \subseteq X$

et $[a, b] \sqcup [a', b'] \stackrel{\text{def}}{=} [\min(a, a'), \max(b, b')]$, $\forall X: \perp \sqcup X \stackrel{\text{def}}{=} X$

Treillis complets

Un **treillis complet** $(X, \sqsubseteq, \sqcup, \sqcap, \perp, \top)$ est un ordre partiel ayant

- 1 une lub $\sqcup S$ pour tout ensemble $S \subseteq X$
- 2 une glb $\sqcap S$ pour tout ensemble $S \subseteq X$
- 3 un plus petit élément $\perp$
- 4 un plus grand élément $\top$

Preuves : * *

- 1 implique 2 car $\sqcap S = \sqcup \{ y \mid \forall x \in S, y \sqsubseteq x \}$
(et 2 implique 1 de même),
- 1 et 2 impliquent 3 et 4 : $\perp = \sqcup \emptyset = \sqcap X$, $\top = \sqcap \emptyset = \sqcup X$

Tout treillis complet est aussi un CPO ;
l'inverse n'est pas vrai.

Exemples de treillis complets

- tout **treillis fini** est complet
- les **intervalles entiers** avec des bornes finies ou **infinies** :

$$(\mathbb{I}, \subseteq, \sqcup, \cap, \perp, [-\infty, +\infty])$$

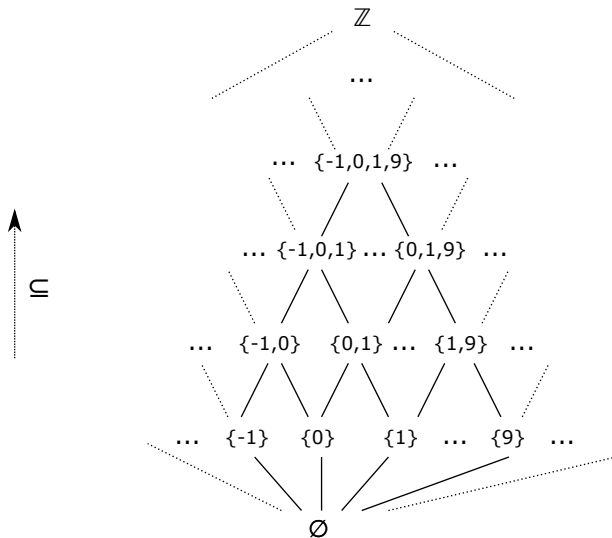
où $\mathbb{I} \stackrel{\text{def}}{=} \{ [a, b] \mid a \in \mathbb{Z} \cup \{-\infty\}, b \in \mathbb{Z} \cup \{+\infty\}, a \leq b \} \cup \{ \perp \}$

et $\sqcup_{i \in I} [a_i, b_i] \stackrel{\text{def}}{=} [\min_{i \in I} a_i, \max_{i \in I} b_i]$.

- $(\mathcal{P}(S), \subseteq, \cup, \cap, \emptyset, S)$ est complet
(voir transparent suivant)

Exemple : les parties d'un ensemble

Exemple : $(\mathcal{P}(\mathbb{Z}), \subseteq, \cup, \cap, \emptyset, \mathbb{Z})$



Fonctions

Une fonction $f : (X_1, \sqsubseteq_1, \sqcup_1, \perp_1) \rightarrow (X_2, \sqsubseteq_2, \sqcup_2, \perp_2)$ est

- **croissante** si : $\forall x, x', x \sqsubseteq_1 x' \implies f(x) \sqsubseteq_2 f(x')$

(en anglais : *monotonic*, qui veut bien dire croissante et non monotone)

- **stricte** si : $f(\perp_1) = \perp_2$

- **continue** entre des CPO si :

pour toute chaîne C de X_1

son image $\{f(c) \mid c \in C\}$ est une chaîne dans X_2

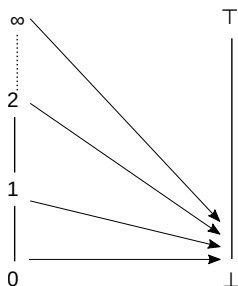
et $f(\sqcup_1 C) = \sqcup_2 \{f(c) \mid c \in C\}$

l'image d'une limite est la limite des images,
donc pas de surprise lors du passage à la limite,
(illustration au transparent suivant)

- **morphisme complet pour $\sqcup$** dans un treillis complet si

$\forall (X_i)_{i \in I}$ famille finie ou infinie d'éléments, $f(\sqcup_{1, i \in I} X_i) = \sqcup_{2, i \in I} f(X_i)$

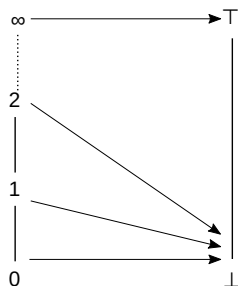
Continuité : illustration



$$\begin{cases} i \mapsto \perp & \text{si } i \in \mathbb{N} \\ \infty \mapsto \perp \end{cases}$$

Fonction continue

pas de surprise à l'infini



$$\begin{cases} i \mapsto \perp & \text{si } i \in \mathbb{N} \\ \infty \mapsto T \end{cases}$$

Fonction non continue

Points fixes

Étant donnée $f : (X, \sqsubseteq) \rightarrow (X, \sqsubseteq)$

- x est un **point fixe** de f si $f(x) = x$
- x est un **pre** point fixe de f si $x \sqsubseteq f(x)$
- x est un **post** point fixe de f si $f(x) \sqsubseteq x$

Il peut y avoir plusieurs points fixes, ou aucun.
Nous notons **lfp** f le plus petit point fixe de f .

Beaucoup de problèmes de mathématique, de physique, mais aussi d'informatique peuvent se réduire à un calcul de point fixe.

Exemple d'utilisation de points fixes

Solutions d'équations **mutuellement récursives**.

Exemple :

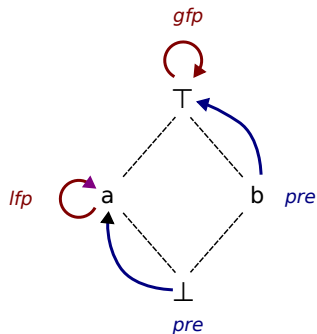
Les solutions de $\begin{cases} x_1 = f(x_1, x_2) \\ x_2 = g(x_1, x_2) \end{cases}$ où x_1, x_2 sont dans un treillis X

sont exactement les **points fixes de $\vec{F}$** dans le treillis $X \times X$, où

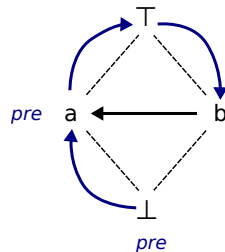
$$\vec{F} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} f(x_1, x_2) \\ g(x_1, x_2) \end{pmatrix}$$

La plus petite solution du système est **lfp $\vec{F}$** $\in X \times X$.

Points fixes : exemples



Fonction **croissante**
avec deux **points fixes**



Fonction **non croissante**
sans point fixe

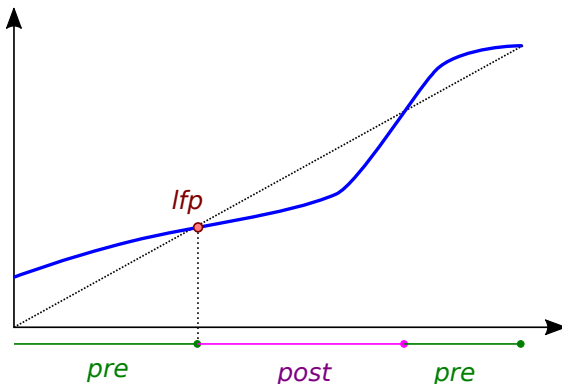
Théorème de point fixe de Tarski

Théorème de Tarski

Si $f : X \rightarrow X$ est **croissante** dans un **treillis complet** X alors $\text{lfp } f$ existe
(en fait, l'ensemble des points fixes de f forme un treillis)

Le théorème précise que :

$$\text{lfp } f = \sqcap \{ x \mid f(x) \sqsubseteq x \} \quad (\text{i.e., le plus petit des post points fixes})$$

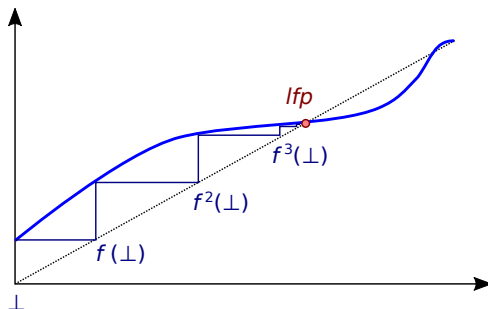


Théorème de point fixe de « Kleene »

Théorème de « Kleene »

Si $f : X \rightarrow X$ est **continue** dans un **CPO** X alors $\text{lfp } f$ existe.

Le théorème précise que : $\{f^n(\perp) \mid n \in \mathbb{N}\}$ est une chaîne et que $\text{lfp } f = \sqcup \{f^n(\perp) \mid n \in \mathbb{N}\}$.



Théorème de point fixe de « Kleene » : preuve [★]_{★★}

Preuve d'une petite généralisation du théorème. . .

Hypothèse : $f : X \rightarrow X$ est continue et $a \sqsubseteq f(a)$.

À prouver : $\text{lfp}_a f = \sqcup \{ f^n(a) \mid n \in \mathbb{N} \}$. (lfp_a est le plus petit point fixe, plus grand que a)

$a \sqsubseteq f(a)$ par hypothèse.

$f(a) \sqsubseteq f(f(a))$ par croissance de f . (toute fonction continue est croissante)

Par récurrence, $\forall n, f^n(a) \sqsubseteq f^{n+1}(a)$.

Donc, $\{ f^n(a) \mid n \in \mathbb{N} \}$ est une chaîne et donc $\sqcup \{ f^n(a) \mid n \in \mathbb{N} \}$ existe.

$f(\sqcup \{ f^n(a) \mid n \in \mathbb{N} \})$

$= \sqcup \{ f^{n+1}(a) \mid n \in \mathbb{N} \}$ (par continuité)

$= a \sqcup (\sqcup \{ f^{n+1}(a) \mid n \in \mathbb{N} \})$ (tous les $f^{n+1}(a)$ sont plus grands que a)

$= \sqcup \{ f^n(a) \mid n \in \mathbb{N} \}$.

Donc, $\sqcup \{ f^n(a) \mid n \in \mathbb{N} \}$ est un point fixe de f .

De plus, tout point fixe plus grand que a doit être plus grand que tous les $f^n(a)$, $n \in \mathbb{N}$.

Donc, $\sqcup \{ f^n(a) \mid n \in \mathbb{N} \} = \text{lfp}_a f$.

Retour à la sémantique des boucles

Sémantique des boucles

$$S[\text{while } c \text{ do } s] R \stackrel{\text{def}}{=} C[\neg c] (\text{lfp } F)$$

$$\text{où } F(X) \stackrel{\text{def}}{=} R \cup S[s] (C[c] X)$$

Justification : $\text{lfp } F$ existe

Nous appliquons les théorèmes de points-fixes vus plus haut avec les hypothèses suivantes :

- $(\mathcal{P}(\mathcal{E}), \subseteq, \cup, \cap, \emptyset, \mathcal{E})$ est un **treillis complet**
c'est donc aussi un CPO
et toutes les séquences croissantes ont une limite
- toutes les fonctions sémantiques sont **continues**
ce sont en fait des morphismes complets pour $\cup$
cela se prouve par induction sur la syntaxe des expressions

Note :

en cas de **boucles imbriquées**,
la sémantique calcule des **points fixes imbriqués**...

Interprétation comme limite de chaîne croissante

$$S[\text{while } c \text{ do } s] R \stackrel{\text{def}}{=} C[\neg c] (\text{lfp } F)$$

$$\text{où } F(X) \stackrel{\text{def}}{=} R \cup S[s] (C[c] X)$$

F applique une itération de la boucle à X

décalant les itérés accumulés d'un cran

puis F ajoute les environnements R avant la boucle, i.e., les itérés au rang 0

Le théorème de Kleene précise que $\text{lfp } F = \bigcup_{n \in \mathbb{N}} F^n(\emptyset)$

- $F^0(\emptyset) = \emptyset$
- $F^1(\emptyset) = R$
environnements avant d'entrer dans la boucle
- $F^2(\emptyset) = R \cup S[s] (C[c] R)$
environnements après zéro ou une itération de boucle
- $F^n(\emptyset)$: environnements après au plus $n - 1$ itérations de boucle
juste avant de tester la condition de sortie,
pour déterminer si une n -ième itération est nécessaire
- $\bigcup_{n \in \mathbb{N}} F^n(\emptyset)$ est bien l'invariant de boucle

Vous pourrez observer les itérations sur des exemples en TME.

Implantation de la sémantique concrète en OCaml

Analyseur

Les sources fournies pour le projet / TME comprennent :

- un *front-end* permettant de lire un code source en texte et de le transformer en [arbre syntaxique](#)
- un [itérateur](#) générique permettant de parcourir l'arbre

L'itérateur est paramétré par un [domaine d'interprétation](#) :

- [dans ce cours](#) : le domaine concret des [ensembles d'environnements](#)
- dans le futur : des domaines abstraits d'analyse statique
- nous utilisons des [foncteurs OCaml](#) :
 - signature [DOMAIN](#) dans [domains/domain.ml](#)
 - implantation de la signature dans [domains/concrete_domain.ml](#)
 - utilisation dans [interpreter/interpreter.ml](#)

Exemple de programme

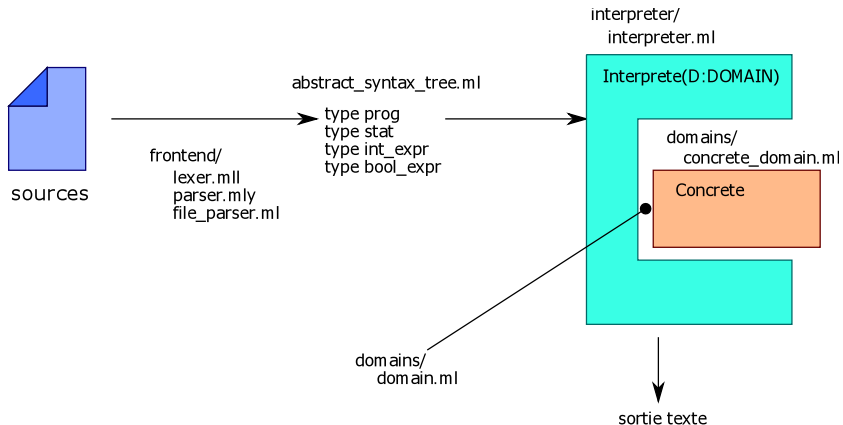
0306_loop_rel.c

```
{  
  int N;  
  int x;  
  N = rand(0,50);  
  x = 0;  
  while (x < N) {  
    print(x,N);  
    x = x + rand(0,3);  
  }  
  assert(x>=N && x<N+3);  
}
```

- blocs introduisant des variables locales
(pas de variables globales)
- type unique : entiers mathématiques $\mathbb{Z}$
- expressions arithmétiques et booléennes
- non-déterminisme avec `rand`
- `while`, `if`, `assert`
- affichage des variables avec `print`

De nombreux autres exemples dans [tests/](#)

Schéma de fonctionnement de l'interprète concret



Technologie du *front-end* : OCamlLex et Menhir [★]_{★★}

lexer.mll

```

{
open Lexing
open Abstract_syntax_tree
open Parser

(* table des mots-clés *)
let kwd_table = Hashtbl.create 10
let _ =
  List.iter (fun (a,b) ->
    Hashtbl.add kwd_table a b)
    [ "int",    TOK_INT;
      "true",   TOK_TRUE;
      "false",  TOK_FALSE;
      ...
    ]
(* utilitaires *)
let space = [' ' '\t' '\r']+
let newline = "\n" | "\n" | "\r\n"
...
(* règles lexicales *)
rule token = parse
| ['a'-'z' 'A'-'Z' '_' ]
| ['a'-'z' 'A'-'Z' '0'-'9' '_' ]* as id
{ try Hashtbl.find kwd_table id
  with Not_found -> TOK_id id }
| "("      { TOK_LPAREN }
| ")"      { TOK_RPAREN }
...

```

parser.mly

```

%{
open Abstract_syntax_tree
%}
(* liste des symboles *)
%token TOK_INT
%token TOK_TRUE
...
(* priorité et associativité des symboles *)
%left TOK_BAR_BAR
%left TOK_AND_AND
...
(* règle racine avec son type *)
%start<Abstract_syntax_tree.prog> file
file: t=list(ext(stat)) TOK_EOF { t }
...
(* règle pour les expressions entières *)
int_expr:
| TOK_LPAREN e=int_expr TOK_RPAREN    { e }
| e=ext(TOK_int)                      { AST_int_const e }
| e=ext(TOK_id)                       { AST_identifieur e }
| o=int_unary_op e=ext(int_expr)
  { AST_int_unary (o,e) }
| e1=ext(int_expr) o=int_binary_op
  e2=ext(int_expr)
  { AST_int_binary (o,e1,e2) }
...

```

Arbre syntaxique abstrait

abstract_syntax_tree.ml

```

type 'a ext = 'a * extent (* ajoute une information de position dans le source *)

type int_unary_op = (* opérateurs unaires entiers *)
  | AST_UNARY_PLUS
  | AST_UNARY_MINUS

type int_binary_op = ... (* opérateurs binaires entiers *)

type int_expr = (* expressions entières *)
  | AST_int_unary  of int_unary_op * (int_expr ext)
  | AST_int_binary of int_binary_op * (int_expr ext) * (int_expr ext)
  | AST_identifieur of var ext
  | AST_int_const  of string ext
  | AST_rand       of (string ext) * (string ext)

type bool_expr = ... (* expressions booléennes *)

type stat = (* instructions *)
  | AST_block    of (((typ * var) ext) list) * ((stat ext) list)
  | AST_assign  of (var ext) * (int_expr ext)
  | AST_if      of (bool_expr ext) * (stat ext) * (stat ext option)
  | AST_while   of (bool_expr ext) * (stat ext)
  | AST_HALT
  | AST_assert  of bool_expr ext
  | AST_print   of (var ext) list

type prog = stat ext list (* programmes *)

```


Bibliothèques : `Zarith` et `Map`

Représentation des entiers :

Les entiers d'OCaml font 31-bit ou 63 bits.

Notre langage utilise des entiers non bornés.

⇒ utilisation d'une bibliothèque d'entiers longs

- module `Z`
- type des entiers : `Z.t`
- opérations classiques : `Z.add`, `Z.sub`, ...
- interface similaire à `Int64`

Représentation des environnements : (voir `domains/concrete_domain.ml`)

Nous avons besoin d'ensembles de fonctions : $\mathcal{P}(\mathbb{V} \rightarrow \mathbb{Z})$.

- représentations de fonctions dans $\mathbb{V} \rightarrow \mathbb{Z}$
 - `module Env = Mapext.Make(String)`
`type env = Z.t Env.t`
 - utilisation du module `Mapext` généralisant `Map` d'OCaml
- pour un ensemble de fonctions, nous utilisons le module `Set`
 - `module EnvSet = Set.Make (struct`
`type t = env`
`let compare = Env.compare Z.compare end)`

Domaine d'interprétation : signature

domains/domain.ml

```

module type DOMAIN =
sig
  type t (* ensemble d'environnements *)

  (* initialisation *)
  val init: unit -> t      (* aucune variable *)
  val bottom: unit -> t    (*  $\emptyset$  *)

  val add_var: t -> var -> t  (* ajout de variable *)
  val del_var: t -> var -> t  (* retrait de variable *)

  val assign: t -> var -> int_expr -> t (*  $V := e$  *)
  val compare: t -> int_expr -> compare_op -> int_expr -> t (*  $e_1 \text{ cmp } e_2$  *)

  val join: t -> t -> t      (*  $\cup$  *)
  val meet: t -> t -> t     (*  $\cap$  *)
  val widen: t -> t -> t
  val subset: t -> t -> bool (*  $\subseteq$  *)
  val is_bottom: t -> bool  (*  $= \emptyset$  *)

  (* affichage *)
  val print: Format.formatter -> t -> var list -> unit
  val print_all: Format.formatter -> t -> unit
end

```

Domaine concret d'interprétation

domains/concrete_domain.ml

```

module Concrete = (struct
  type t = EnvSet.t                (*  $\mathcal{P}(\mathcal{E})$  *)
  module ValSet = Set.Make(Z)      (*  $\mathcal{P}(\mathbb{Z})$  *)

  let int_map (f:Z.t -> Z.t) (s:ValSet.t) : ValSet.t = (*  $\{f(x) \mid x \in s\}$  *)
    ValSet.fold (fun x acc -> ValSet.add (f x) acc) s ValSet.empty

  let rec eval_expr (e:int_expr) (m:env) : ValSet.t = (*  $E[e] m$  *)
    match e with
    | AST_int_unary (op,(e1,_)) ->
      let s1 = eval_expr e1 m in
      int_map
        (match op with
         | AST_UNARY_PLUS -> fun x -> x
         | AST_UNARY_MINUS -> Z.neg
        ) s1
    | ...
    (*  $S[v \leftarrow e] m = \{\rho[v \mapsto x] \mid \rho \in m, x \in E[e] \rho\}$  *)
  let assign (m:t) (v:string) (e:int_expr) : t =
    EnvSet.fold
      (fun env acc ->
        let s = eval_expr e env in
        ValSet.fold (fun v acc -> EnvSet.add (Env.add v v env) acc) s acc
      ) m EnvSet.empty

  let join m1 m2 = EnvSet.union m1 m2 (*  $\cup$  *)
end: DOMAIN)

```

Itérateur

interpreter/interpreter.ml

```
module Interprete(D : DOMAIN) = struct
  type t = D.t (*  $\mathcal{P}(\mathcal{E})$  *)

  (*  $C[e] a$  si  $r=true$ ,  $C[\neg e] a$  si  $r=false$  *)
  let filter (a:t) (e:bool_expr ext) (r:bool) : t =
    let rec doit a (e,x) r = match e with
    | AST_bool_unary (AST_NOT, e) -> doit a e (not r)
    | AST_bool_binary (AST_AND, e1, e2) ->
      (if r then D.meet else D.join) (doit a e1 r) (doit a e2 r)
    | ...
    | AST_compare (cmp, (e1,_), (e2,_)) ->
      let inv = function
      | AST_EQUAL      -> AST_NOT_EQUAL
      | AST_NOT_EQUAL  -> AST_EQUAL
      | AST_LESS       -> AST_GREATER_EQUAL
      | AST_LESS_EQUAL -> AST_GREATER
      | AST_GREATER    -> AST_LESS_EQUAL
      | AST_GREATER_EQUAL -> AST_LESS
      in
      let cmp = if r then cmp else inv cmp in
      D.compare a e1 cmp e2
    in
    doit a e r
```

Itérateur (suite)

interpreter/interpreter.ml

```
let rec eval_stat (a:t) ((s,ext):stat ext) : t = (* S[s] a *)
  match s with
  | AST_assign ((i,_),(e,_)) -> D.assign a i e

  | AST_if (e,s1,Some s2) ->
    let t = eval_stat (filter a e true ) s1 in
    let f = eval_stat (filter a e false) s2 in
    D.join t f

  | AST_assert e ->
    a

  | AST_block (decl,inst) ->
    let a = List.fold_left (fun a ((_,v),_) -> D.add_var a v) a decl in
    let a = List.fold_left eval_stat a inst in
    List.fold_left (fun a ((_,v),_) -> D.del_var a v) a decl

  | AST_while (e,s) ->
    let rec fix (f:t -> t) (x:t) : t =
      let fx = f x in
      if D.subset fx x then fx
      else fix f fx
    in
    let f x = D.join a (eval_stat (filter x e true) s) in
    let inv = fix f a in
    filter inv e false
```